



¿Cómo realizo una traza?

Las **trazas** son indispensables para cualquier programador. Sirven para verificar, informalmente, la correctitud del código, buscar potenciales errores semánticos e incluso nos pueden ayudar a visualizar optimizaciones de las soluciones. Además, para los estudiantes son una excelente herramienta para poder comprender cómo funciona el uso de la memoria.

Particularmente, en RPA, los exámenes se rinden en papel, por lo tanto las trazas van a ser sus mejores aliadas para poder encontrar posibles errores en su código. 😊

Entonces, a la hora de programar les sugerimos que respeten los siguientes pasos:

1. Leer detenidamente el problema a resolver y comprenderlo en su totalidad;
2. Bosquejar un algoritmo determinando datos de entrada y de salida;
3. Programarlo en Pascal;
4. Realizar una **traza**;
5. Pasarlo en el entorno Lazarus y ejecutarlo para casos de prueba seleccionados cuidadosamente;
6. Verificar que los resultados de la traza y de la ejecución del programa coincidan.

Ahora bien, ¿Cómo realizamos una traza?. Realmente no existen estándares a la hora de trazar programas, de hecho a medida que los programadores se van formando cada uno adopta estilos personales. En RPA estandarizamos las trazas utilizando tablas donde se pueden ver los valores de las variables a medida que avanza la ejecución del programa y además para las estructuras de control con condición (if, while, repeat-until) utilizamos un evaluador de expresión (EE) para poder determinar los valores de verdad de la condición durante la ejecución.

A continuación les presentamos dos ejemplos de trazas completas:

Programa 1:

```
program problemal;  
  var a,b,c: integer;  
begin  
  a:=20;  
  b:=10;  
  c:=100;  
  if(a>10) then  
    a:=a-1;  
  if(b=0) then  
    b:=b-1;  
  if(c>20) then  
    c:=c-1;  
end.
```

a	b	c
20	10	100
19		99

Primero lo que debemos hacer es identificar cuáles son las variables de nuestro programa. En este caso son **a**, **b** y **c**.

Por otro lado debemos identificar cuantas estructuras de control con condición tenemos, en este caso son las tres sentencias if.

Con todo esto ya determinado podemos pasar a la construcción de la tabla donde visualizaremos la evolución del valor de las variables y del evaluador de expresión (EE) donde visualizaremos los valores de verdad producto de la evaluación de cada una de las condiciones de las estructuras de control determinadas anteriormente

EE

a>10→20>10→TRUE

b=0→10=0→FALSE

c>20→100>20→TRUE

Programa 2:

```
program problema2;  
var a, b: integer;  
begin  
  a:=8;  
  b:=10;  
  while (a<b) do  
    a:= a + 1;  
  end.
```

En este caso las variables del programa son **a** y **b**, y tenemos una única estructura de control con condición: el **while**.

Veamos como construir la traza paso a paso:

1. Como tenemos dos variables, vamos a armar una tabla con dos columnas, una para a y otra para b.

Comenzamos a ejecutar el bloque ejecutable del programa. Encontramos dos asignaciones. Con la primera se asigna el valor 8 a la variable **a**, y con la segunda se asigna el valor 10 a la variable **b**.

a	b
8	10

2. **while** (a<b) **do**

Al encontrar un while tenemos que acudir al Evaluador de Expresiones (EE) para evaluar la condición: $a < b \rightarrow 8 < 10 \rightarrow \text{TRUE}$

Como la evaluación de la condición da VERDADERO, entramos al ciclo while. y ejecutamos la asignación que está dentro del while. Se suma 1 al valor guardado en la variable **a** ($a := a + 1$), y la variable b no cambia.

EE

$a < b \rightarrow 8 < 10 \rightarrow \text{TRUE}$

a	b
8	10
9	

3. **while** a<b **do**

Nuevamente tenemos que evaluar la condición del while, para ver si tenemos que seguir repitiendo. Evaluador de expresiones (EE): $a < b \rightarrow 9 < 10 \rightarrow \text{TRUE}$

Entra el ciclo while y suma 1 al valor guardado en la variable **a** ($a := a + 1$), la variable b no cambia.

EE

$a < b \rightarrow 8 < 10 \rightarrow \text{TRUE}$

$a < b \rightarrow 9 < 10 \rightarrow \text{TRUE}$

a	b
8	10
9	
10	

1. **while** $a < b$ **do**

Siempre que terminamos de ejecutar el bloque del while, tenemos que volver a evaluar la condición del while. Evaluador de expresiones (EE): $a < b \rightarrow 10 < 10 \rightarrow \text{FALSE}$

EE

$a < b \rightarrow 8 < 10 \rightarrow \text{TRUE}$

$a < b \rightarrow 9 < 10 \rightarrow \text{TRUE}$

$a < b \rightarrow 10 < 10 \rightarrow \text{FALSE}$

a	b
8	10
9	
10	

Como esta vez da FALSO, el while termina y el programa finaliza con a valiendo 10 y b valiendo tambien 10.